hashlock.

Security Audit

OObleck (DeFi)

October 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

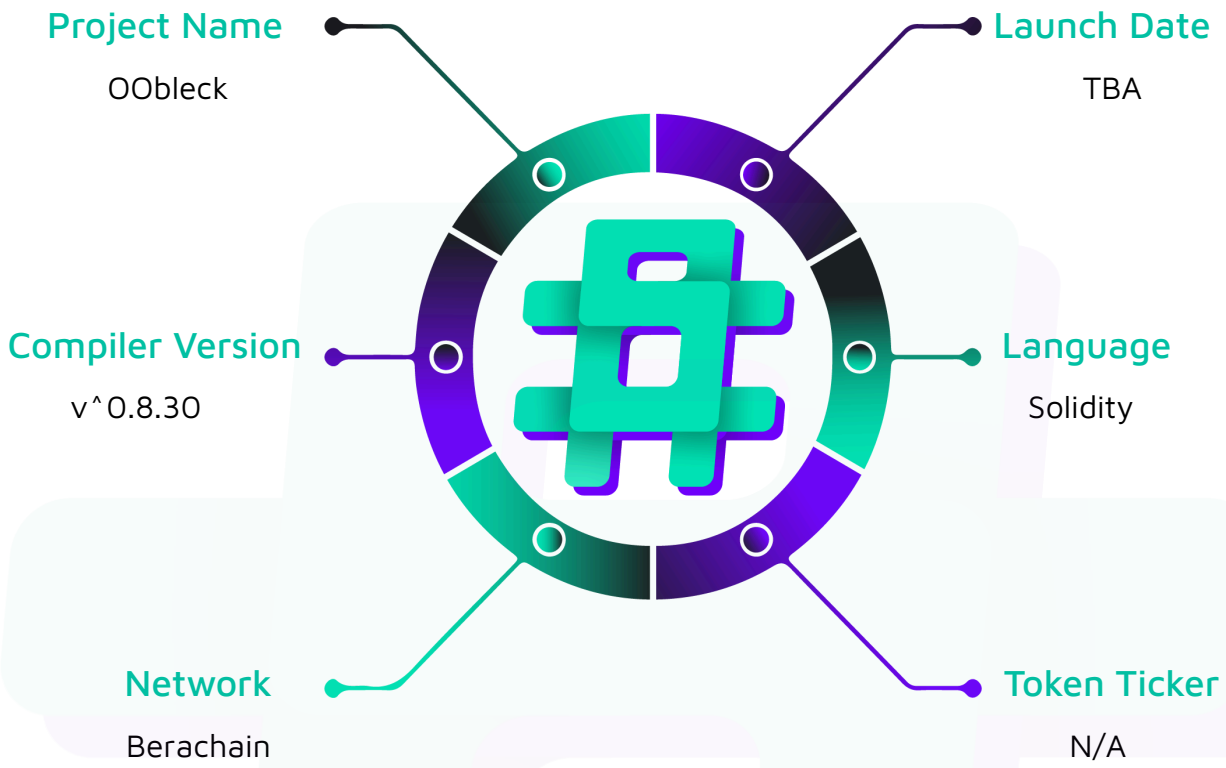
The OObleck team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

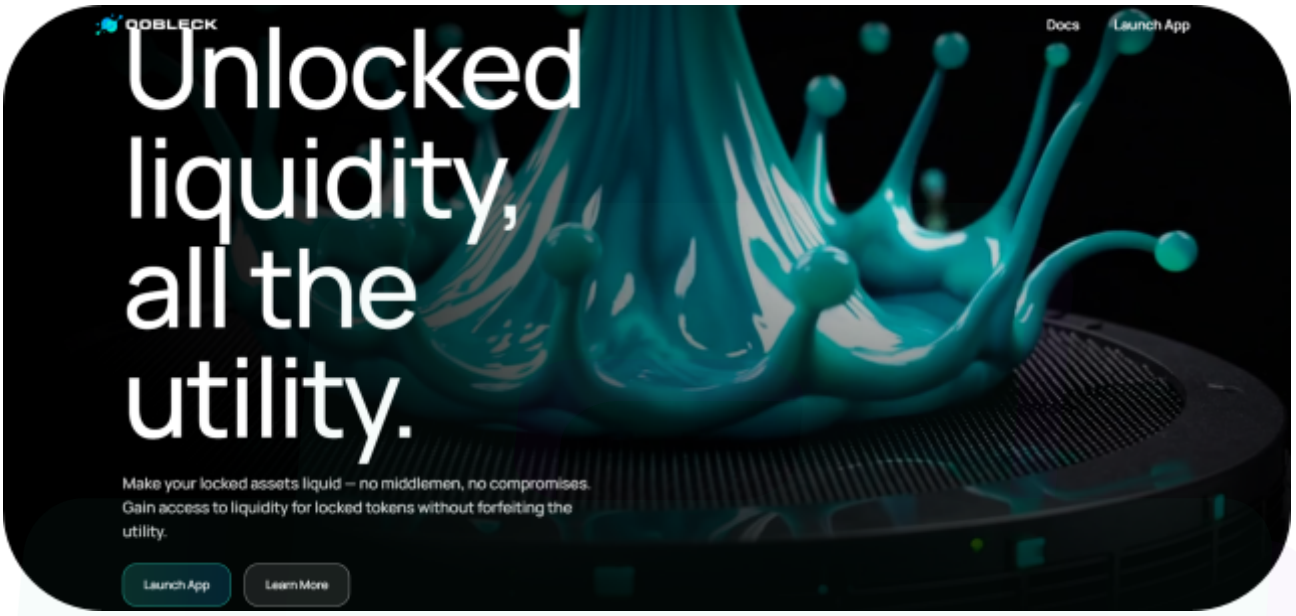
Oobleck bridges the gap between utility and liquidity. Your locked assets stay productive while becoming portable and market-ready, redefining how capital moves in the new on-chain economy.

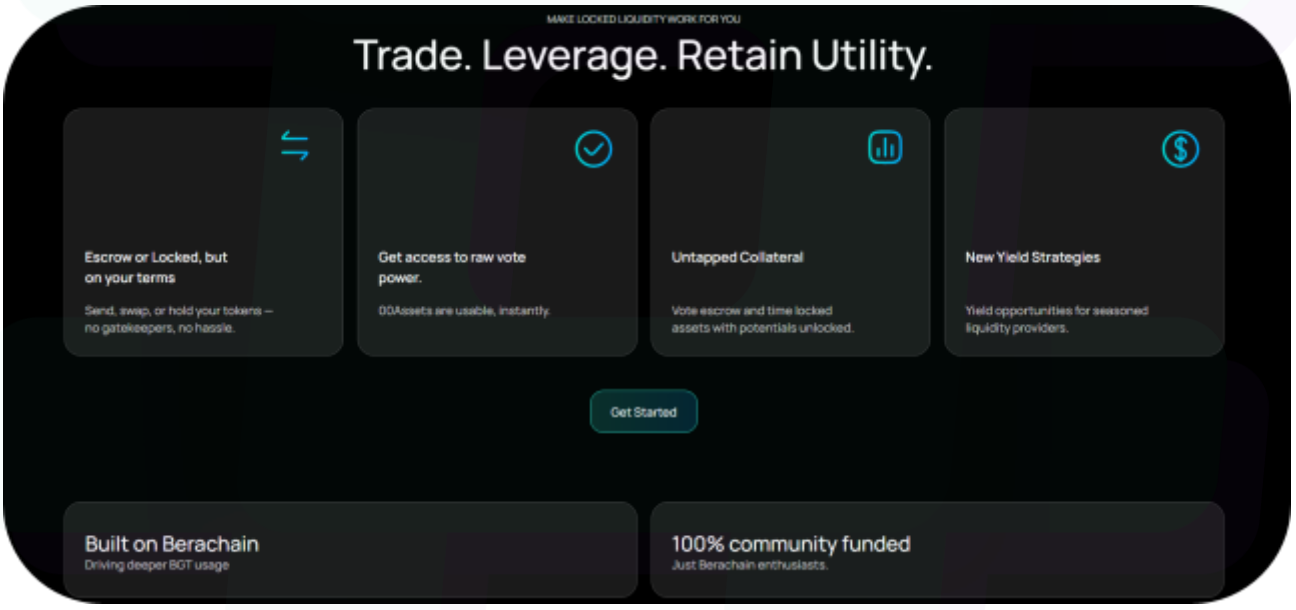
Project Name: OObleck
Project Type: Defi
Compiler Version: ^0.8.30
Website: https://oobleck.xyz/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the Solidity code within the OObleck project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**OObleck Protocol Smart Contracts**
Platform	**Berachain / Solidity**
Audit Date	**October, 2025**
Contract 1	OOBGTERC6551Account.sol
Contract 2	OOBGTNFT.sol
Contract 3	ClaimOperator.sol
Contract 4	FeeDistributor.sol
Contract 5	OOBGTMarketplace.sol
Audited GitHub Commit Hash	57af9413580ddcf8a113789169730da8e3f8bab6
Fix Review GitHub Commit Hash	d3f6852b912f6896da795cbaabff8e1a57533dfe

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High severity vulnerabilities

1 Low severity vulnerabilities

1 Gas Optimisations

1 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
OOBGTNFT.sol - Allows users to: - Mint new OOBGT NFTs with associated TBAs - Burn NFTs when BGT balance is zero - Batch redeem BGT from multiple NFTs with fees - Approve operators when BGT threshold is met - Batch approve NFTs for marketplace listing	**Contract achieves this functionality.**
ClaimOperator.sol - Allows users to: - Claim partial rewards from individual contracts - Send claimed rewards directly to TBA recipients - Process claims with validation for listed NFTs	**Contract achieves this functionality.**
FeeDistributor.sol - Allows owner to: - Propose and execute fee recipient changes (with timelock) - Propose and execute fee percentage changes (with timelock) - Recover tokens and native currency - Cancel pending timelock actions	**Contract achieves this functionality.**
OOBGTMarketplace.sol - Allows users to: - Create and cancel sell offers for NFTs - Accept offers by purchasing NFTs - Place collection bids with BGT pricing - Accept bids to sell NFTs - Accept bids for multiple NFTs at once	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the OObleck project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the OObleck project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] ClaimOperator#_assertRecipientIsTBA - TBA Validation Can Be Bypassed by Malicious Contracts

Description

The `ClaimOperator` contract's `TBA` validation is weak, as any contract can mimic the `token()` function. This allows attackers to pass the check and redirect rewards to malicious addresses.

Vulnerability Details

The `_assertRecipientIsTBA` function performs weak checks that can be easily spoofed. It doesn't confirm the recipient was created through the official `ERC-6551` registry, allowing fake contracts to pass as valid `TBAs`.

Impact

Attackers can claim rewards to fake `TBA` contracts they control, bypassing the intended security model. This allows theft of validator rewards by directing them to attacker-controlled contracts.

Recommendation

Implement proper `TBA` validation by verifying against the `ERC-6551` registry. The validation should compute the expected TBA address using the registry's `account()` function with the provided `implementation`, `salt`, `chainId`, `tokenContract`, and `tokenId`, then compare it against the `recipient` address. Only if they match should the recipient be considered a valid `TBA`.

Status

Resolved

[H-02] OOBGTERC6551Account#_checkValidFn - Function Selector Blocking Can Be Bypassed

Description

The `TBA` implementation attempts to prevent token transfers and approvals by blocking specific function selectors. However, the blocklist is incomplete and only covers a subset of standard `ERC20/ERC721` functions. Numerous alternative functions and token standards can bypass these restrictions, allowing users to transfer or approve tokens despite the intended security controls.

Vulnerability Details

Possible bypasses include `increase/decreaseAllowance()`, `ERC1155` functions, custom transfer functions in non-standard tokens, and any future `ERC` standards. The blocklist approach is inherently fragile and incomplete.

Impact

The security restrictions meant to protect `TBA` assets can be bypassed. Users could grant approvals to malicious contracts using `increase/decreaseAllowance()` or move assets out of the `TBA` when they shouldn't be able to. This undermines the `BGT` locking mechanism and marketplace restrictions, potentially allowing users to list `NFTs` and immediately drain their `BGT`, or manipulate locked states.

Recommendation

Implement a whitelist approach instead of a blocklist. Create a mapping of explicitly allowed function selectors and only permit those operations. Alternatively, implement a more sophisticated access control system that validates the target contract and function combination, or restrict execution to a predefined set of protocol contracts.

Status

Resolved

[H-03] OOBGTNFT#setApprovalForAll - Unbounded Loop in setApprovalForAll Causes Denial of Service

Description

The `setApprovalForAll` function contains an unbounded loop that iterates over all tokens ever minted in the collection, not just the tokens owned by the caller. As the total number of minted tokens grows, the gas cost of this function increases linearly, eventually exceeding block gas limits and making the function permanently unusable.

Vulnerability Details

```
function setApprovalForAll(address operator, bool approved) public override {

    uint256 nextTokenId = _nextTokenId;

    if (approved) {

        for (uint256 tokenId = 0; tokenId < nextTokenId; tokenId++) {
```

For each iteration, the function performs multiple operations including existence checks, ownership checks, external calls to get `TBA` addresses, and token balance queries.

Impact

Users will be unable to grant operator approvals once the collection grows beyond a certain size, effectively breaking marketplace integrations, batch transfers, and any other functionality requiring operator approvals.

Recommendation

Remove the automatic `BGT` locking mechanism from `setApprovalForAll` entirely. The BGT lock state should be managed separately through on-demand locking where users explicitly lock `BGT` before listing, or by keeping the locking logic only in the single-token `approve()` function. Alternatively, implement lazy evaluation where `BGT` requirements are checked at the time of transfer rather than at approval time.

Status

Resolved

Low

[L-01] OOBGTNFT - Batch Operations Use All-or-Nothing Pattern Unnecessarily

Description

The batch operation functions like `batchQueueBoostByOwner` use `require` statements that cause the entire transaction to revert if any single operation fails. While atomic behavior has some benefits, it creates poor user experience when one invalid token causes all valid operations to fail.

Recommendation

Use try-catch patterns or conditional checks to continue processing on failures, similar to how `batchRedeemBgt` is implemented. This provides better `UX` while still returning the count of successful operations.

Status

Resolved

Gas

[G-01] OOBGTNFT - Storage Variables Read in Loops Waste Gas

Description

Several batch operation functions repeatedly read storage variables like `bgtToken` within loops, incurring multiple expensive `SLOAD` operations.

Recommendation

Cache storage variables in memory before loops to reduce gas costs.

Status

Resolved

QA

[Q-01] OOBGTNFT - Missing Event Emission for State Changes

Description

The `setListed` function modifies critical state but doesn't emit an event, making it difficult for off-chain indexers and frontends to track `NFT` listing status changes.

Recommendation

Add a `ListedStatusChanged` event and emit it whenever the listing status changes.

Status

Resolved

Centralisation

The OObleck project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the OObleck project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au